

Introduction à Hadoop.

Cas d'étude: HDFS.

Réalisé par:

M^{elle}.GUETARNI Hala

Université Hassiba Ben Bouali - Chlef

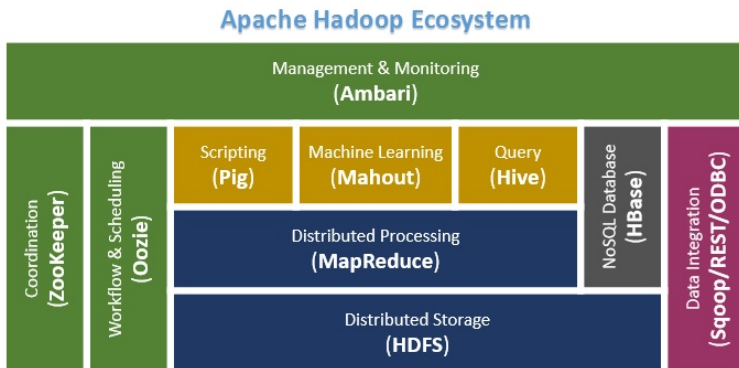
November 20, 2023

Qu'est ce que Hadoop?:

- ▶ Projet de haut niveau Apache, mise en œuvre à source ouverte (open source) pour un calcul et un stockage de données fiables, évolutifs et distribués.
- ▶ Il s'agit d'une architecture flexible et de haute disponibilité pour le calcul et le traitement de données à grande échelle.
- ▶ Conçu pour répondre à la question : "Comment traiter les données volumineuses à un coût et dans des délais raisonnables ?"

Quels sont les outils de framework Hadoop?:

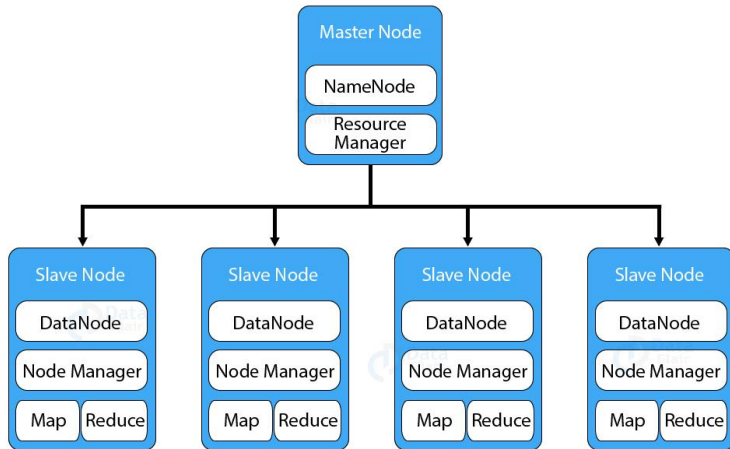
- Dans cette figure, vous allez découvrir les meilleurs outils Hadoop Big Data disponibles sur le marché.



Qu'est ce que HDFS?:

- ▶ HDFS est un système de fichiers distribué "Hadoop Distributed File System". Qui donne un accès haute-performance aux données réparties dans des clusters Hadoop.
- ▶ Dans Hadoop, les données sont stockées dans le système de fichiers distribués Hadoop (HDFS), qui est spécifiquement conçu pour gérer de grands ensembles de données sur plusieurs nœuds d'un cluster.
- ▶ Lorsque HDFS collecte des données, le système divise les informations en blocs et les distribue sur plusieurs nœuds du cluster.
- ▶ Le système de fichiers copie chaque élément de données plusieurs fois, distribue les copies à chaque nœud et place au moins une copie sur un autre serveur du cluster.
- ▶ Cela permet aux données d'être stockées sur le nœud défaillant d'être trouvé ailleurs dans le cluster.
- ▶ HDFS dans Hadoop offre une tolérance aux pannes et une haute disponibilité à la couche de stockage et aux autres périphériques présents dans ce cluster Hadoop. Nœuds de stockage de données dans HDFS.

Architecture de HDFS sous forme distribuée:



L'architecture de HDFS - Suite:

Dans un cluster, où les données et les services sont stockées sur plusieurs machines différentes, HDFS fonctionne selon un principe maître/esclaves classique: les données y sont stockées sur les datanodes (esclaves) tandis que les localisations des blocs de données sont répertoriées par le namenode (maître).

► NameNode (Maître):

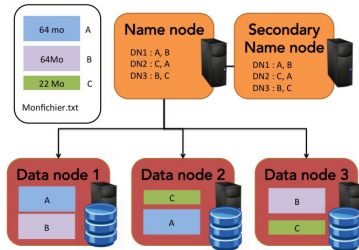
- NameNode fonctionne comme un maître dans un cluster Hadoop qui guide le Datanode (esclaves).
- NameNode s'occupe de la division des données sur l'ensemble des datanodes et il s'occupe également de savoir où se trouve chaque partie du fichier.
- C'est lui également qui va avoir une visibilité sur l'ensemble du cluster.

► DataNodes (Esclaves):

- Nœud HDFS qui stocke les données dans le système de fichiers Hadoop.
- Une grappe HDFS peut avoir plusieurs DataNodes comprenant des données reproduites entre eux.

L'architecture de HDFS - Suite:

- Chaque fichier est divisé en blocs avec une taille maximale fixe. Par défaut, cette taille est de 64 Mo. Par exemple, un fichier de 150 Mo est divisé en trois blocs: deux blocs de 64 Mo et un bloc de 22 Mo. Ces blocs sont répartis de manière redondante sur différents nœuds de données. Le nœud de nom sait comment le fichier est partitionné et dans quels nœuds de données ces blocs sont stockés.

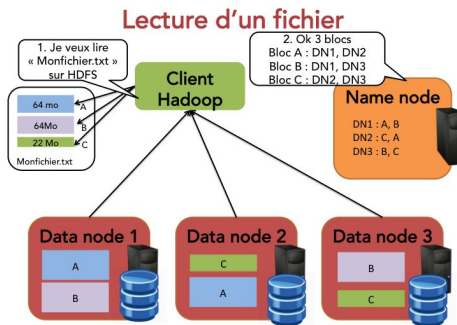


- Dans le schéma ci-dessus, les blocs de données sont stockés avec un taux de réplication de 2 : en fonctionnement nominal, chaque bloc est stocké sur deux datanodes différents.

Lecture / Ecriture d'un fichier:

- ▶ Les datanodes ont très peu d'intelligence et ils ne servent qu'à stocker les données.
- ▶ Les adresses des blocs ainsi que les noms des fichiers sont tous stockés par le namenode, dont le rôle est critique.
- ▶ Pour illustrer le rôle du namenode et des datanodes, détaillons ce qui se passe lorsqu'un client (nommé User_1) veut lire un fichier stocké dans HDFS :

Lecture d'un fichier :

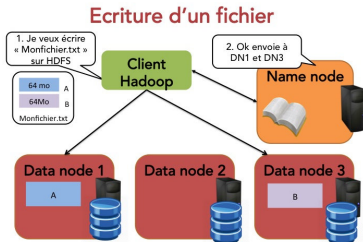


Les étapes de la lecture d'un fichier:

- ▶ User_1 indique au namenode qu'il souhaite lire un fichier.
- ▶ Le name node indique à user_1 la taille du fichier ainsi que les différents data nodes contenant les blocs qui composent ce fichier.
- ▶ User_1 récupère chacun des blocs sur l'un des data nodes.
- ▶ Si un des datanodes est indisponible, User_1 en contacte un autre.

Ecriture d'un fichier :

De la même manière, si un autre client (nommé User_2) veut écrire un fichier, voilà ce qui se passe :



Les étapes de l'écriture sont:

- ▶ User_2 indique au namenode qu'il souhaite écrire un bloc.
- ▶ Le namenode indique à User_2 le datanode à contacter.
- ▶ User_2 envoie le bloc au datanode.
- ▶ Les datanodes répliquent les blocs entre eux.
- ▶ Le cycle se répète pour le bloc suivant.

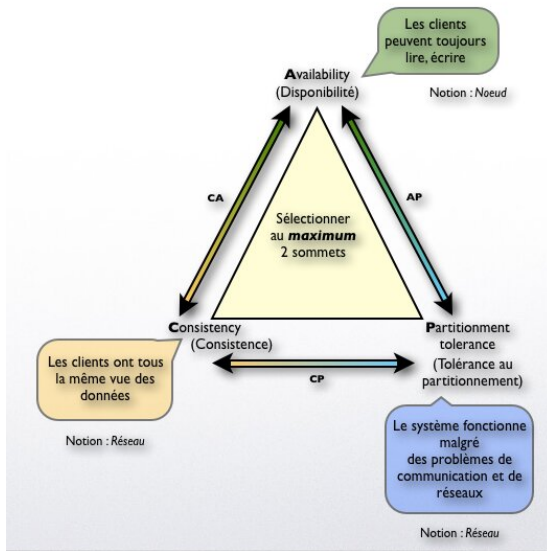
HDFS et le théorème CAP:

Mais alors, que se passe-t-il si User_1 cherche à lire le fichier juste après qu'il a été modifié par User_2 ? Est-ce qu'User_1 va bien obtenir les données les plus récentes ? En particulier, que se passe-t-il en cas de panne d'un des datanodes ? Pour répondre à cette question, il faut comprendre comment se place HDFS sur le triangle CAP.

Avant d'entrer dans les détails, il est nécessaire de savoir qu'est ce qu'un Théorème CAP?

- Le théorème CAP applique une logique similaire aux systèmes distribués, à savoir qu'un système distribué ne peut fournir que deux des trois caractéristiques souhaitées : cohérence, disponibilité (availability) et tolérance au partitionnement (en anglais, le C, le A et le P de CAP).

Schéma explicatif du théorème CAP:



Explication de CAP dans le théorème CAP:

► Cohérence (EN: Consistency):

- La cohérence signifie que tous les clients voient les mêmes données en même temps, quel que soit le nœud auquel ils se connectent.
- Pour que cela se produise, chaque fois que des données sont écrites sur un nœud, elles doivent être immédiatement transmises ou copiées sur tous les autres nœuds du système avant que l'écriture ne soit considérée comme réussie .

► Disponibilité (EN: Availability):

- La disponibilité signifie qu'un client effectuant une demande de données recevra une réponse, même si un ou plusieurs nœuds sont en panne.
- Voici une autre façon d'exprimer ce concept: tous les nœuds actifs du système distribué renvoient des réponses valides à toutes les requêtes, sans exception.

► Tolérance au partitionnement (EN: Partition Tolerance):

- Un partitionnement est une rupture de communication au sein d'un système distribué, une perte ou un retard temporaire de connexion entre deux nœuds.
- La tolérance au partitionnement signifie que la grappe (cluster) doit continuer à fonctionner quel que soit le nombre d'interruptions entre les nœuds du système.

HDFS et CAP:

- ▶ Le théorème CAP est conçu pour les systèmes de fichiers distribués (ensemble de nœuds interconnectés).
- ▶ Dans les systèmes distribués, la cohérence, la disponibilité et la tolérance de partition existent dans une relation mutuellement dépendante. Vous ne pouvez pas avoir les 3, par exemple, si vous choisissez une cohérence stricte, vous devez indiquer la disponibilité, donc choisissez-en deux.
- ▶ **CT (Cohérence/Tolérance au partitionnement (EN: CP)):**
 - ▶ Les systèmes CT privilégient la cohérence et la tolérance aux partitions par rapport à la disponibilité. Ils garantissent que tous les nœuds du système ont la même vue cohérente des données, même en présence de partitions du réseau. Pour ce faire, les systèmes de CP utilisent souvent des techniques telles que les algorithmes de consensus distribués, comme les algorithmes Paxos ou Raft. Ces algorithmes garantissent que tous les nœuds sont d'accord sur l'ordre et la cohérence des mises à jour avant qu'elles ne soient appliquées. Toutefois, si une partition se produit et qu'une minorité de nœuds est isolée, le système sacrifie la disponibilité jusqu'à ce que la partition soit résolue.

HDFS et CAP - Suite:

► DT (Disponibilité/Tolérance au partitionnement (EN: AP)):

- Les systèmes DT se concentrent sur la haute disponibilité et la tolérance aux partitions, même au détriment de solides garanties de cohérence. Ils donnent la priorité à la réactivité et veillent à ce que le système reste disponible pour les clients, même si cela implique de tolérer des incohérences temporaires dans les données. Les systèmes DT utilisent souvent des techniques telles que la réplication et la cohérence éventuelle pour gérer les partitions et maintenir la disponibilité. Ces systèmes peuvent continuer à fonctionner pendant les partitions du réseau en autorisant des mises à jour simultanées et en résolvant les conflits de manière asynchrone. Parmi les exemples de systèmes DT, citons DynamoDB et Riak d'Amazon.

HDFS et CAP - Suite:

► CD (Cohérence/Disponibilité (EN: CA)):

- Les systèmes CD trouvent un équilibre entre la cohérence et la disponibilité. Ils visent à fournir des garanties de cohérence et une haute disponibilité, mais pas au même niveau que les systèmes CT ou DT.
- Les systèmes de CD choisissent souvent d'assouplir les garanties de cohérence en faveur d'une meilleure disponibilité. Ils utilisent des techniques telles que la réplication optimiste, où les conflits sont détectés et résolus après coup, ce qui garantit la cohérence du système. La base de données Spanner de Google est un exemple de système de CD.

Conclusion:

- ▶ Il est important de noter que le théorème CAP suppose que l'apparition de partitions de réseau est inévitable. Dans les systèmes distribués pratiques, les partitions de réseau peuvent se produire et se produisent effectivement en raison de divers facteurs. Les architectes doivent tenir compte des exigences et des compromis propres à leur domaine d'application et faire un choix conscient lorsqu'ils conçoivent des systèmes distribués.
- ▶ Le théorème CAP fournit des indications précieuses sur les compromis impliqués dans la conception des systèmes distribués. Bien qu'il affirme qu'il est impossible d'obtenir simultanément une forte cohérence, une haute disponibilité et une tolérance aux partitions, les architectes peuvent prendre des décisions éclairées basées sur les besoins spécifiques de leurs applications. En comprenant le théorème CAP et ses implications, les concepteurs de systèmes peuvent naviguer dans les complexités des systèmes distribués et trouver un équilibre approprié entre la cohérence, la disponibilité et la tolérance aux pannes.

” Merci de votre attention ”